

Course overview

What is this course?

- Intermediate-level survey course.
- Programming and problem solving, with applications.
- **Algorithm:** method for solving a problem.
- **Data structure:** method to store information.

topic	data structures and algorithms	
data types	stack, queue, bag, union-find, priority queue	part 1
sorting	quicksort, mergesort, heapsort, radix sorts	
searching	BST, red-black BST, hash table	
graphs	BFS, DFS, Prim, Kruskal, Dijkstra	part 2
strings	KMP, regular expressions, TST, Huffman, LZW	
advanced	B-tree, suffix array, maxflow	

Why study algorithms?

Their impact is broad and far-reaching.

Internet. Web search, packet routing, distributed file sharing, ...

Biology. Human genome project, protein folding, ...

Computers. Circuit layout, file system, compilers, ...

Computer graphics. Movies, video games, virtual reality, ...

Security. Cell phones, e-commerce, voting machines, ...

Multimedia. MP3, JPG, DivX, HDTV, face recognition, ...

Social networks. Recommendations, news feeds, advertisements, ...

Physics. N-body simulation, particle collision simulation, ...

⋮

Why study algorithms?

They may unlock the secrets of life and of the universe.

Computational models are replacing math models in scientific inquiry.

$$\begin{aligned} E &= mc^2 \\ F &= ma \qquad F = \frac{Gm_1m_2}{r^2} \\ \left[-\frac{\hbar^2}{2m} \nabla^2 + V(r) \right] \Psi(r) &= E \Psi(r) \end{aligned}$$

20th century science
(formula based)

```
for (double t = 0.0; true; t = t + dt)
  for (int i = 0; i < N; i++)
  {
    bodies[i].resetForce();
    for (int j = 0; j < N; j++)
      if (i != j)
        bodies[i].addForce(bodies[j]);
  }
```

21st century science
(algorithm based)

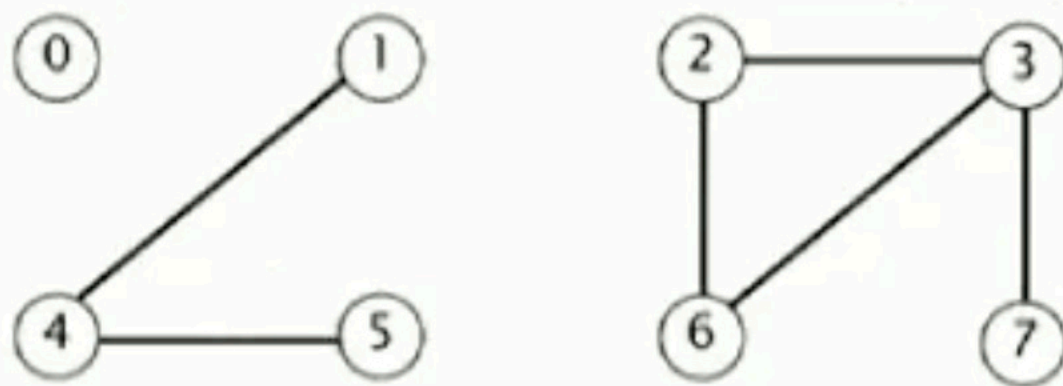
“ Algorithms: a common language for nature, human, and computer. ” — Avi Wigderson

Modeling the connections

We assume "is connected to" is an equivalence relation:

- Reflexive: p is connected to p .
- Symmetric: if p is connected to q , then q is connected to p .
- Transitive: if p is connected to q and q is connected to r , then p is connected to r .

Connected components. Maximal **set** of objects that are mutually connected.

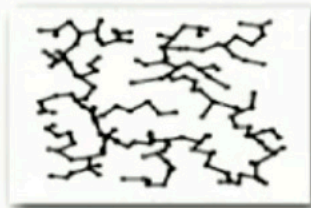
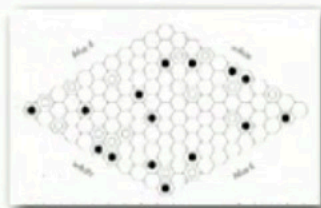


{ 0 } { 1 4 5 } { 2 3 6 7 }

3 connected components

Union-find applications

- Percolation.
- Games (Go, Hex).
- ✓ Dynamic connectivity.
 - Least common ancestor.
 - Equivalence of finite state automata.
 - Hoshen-Kopelman algorithm in physics.
 - Hinley-Milner polymorphic type inference.
 - Kruskal's minimum spanning tree algorithm.
 - Compiling equivalence statements in Fortran.
 - Morphological attribute openings and closings.
 - Matlab's `bwlabel()` function in image processing.



Subtext of today's lecture (and this course)

Steps to developing a usable algorithm.

- Model the problem.
- Find an algorithm to solve it.
- Fast enough? Fits in memory?
- If not, figure out why.
- Find a way to address the problem.
- Iterate until satisfied.

The scientific method.

Mathematical analysis.

Dynamic connectivity

Given a set of N objects.

- **Union command:** connect two objects.
- **Find/connected query:** is there a path connecting the two objects?

```
union(4, 3)
```

```
union(3, 8)
```

```
union(6, 5)
```

```
union(9, 4)
```

```
union(2, 1)
```

```
connected(0, 7) ✗
```

```
connected(8, 9) ✓
```

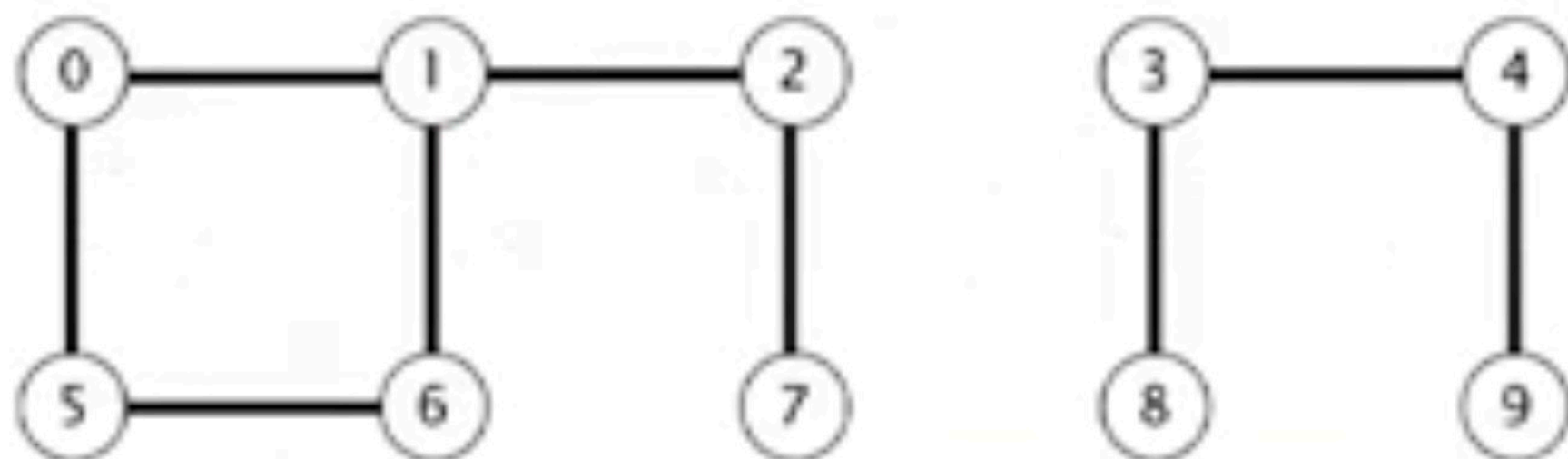
```
union(5, 0)
```

```
union(7, 2)
```

```
union(6, 1)
```

```
union(1, 0)
```

```
connected(0, 7) ✓
```



Modeling the objects

Applications involve manipulating objects of all types.

- Pixels in a digital photo.
- Computers in a network.
- Friends in a social network.
- Transistors in a computer chip.
- Elements in a mathematical set.
- Variable names in Fortran program.
- Metallic sites in a composite system.

Modeling the objects

Applications involve manipulating objects of all types.

- Pixels in a digital photo.
- Computers in a network.
- Friends in a social network.
- Transistors in a computer chip.
- Elements in a mathematical set.
- Variable names in Fortran program.
- Metallic sites in a composite system.

When programming, convenient to name objects 0 to $N - 1$.

- Use integers as array index.
- Suppress details not relevant to union-find.



can use symbol table to translate from site names to integers: stay tuned (Chapter 3)

Modeling the connections

We assume "is connected to" is an equivalence relation:

- Reflexive: p is connected to p .
- Symmetric: if p is connected to q , then q is connected to p .
- Transitive: if p is connected to q and q is connected to r , then p is connected to r .

Quick-find [eager approach]

Data structure.

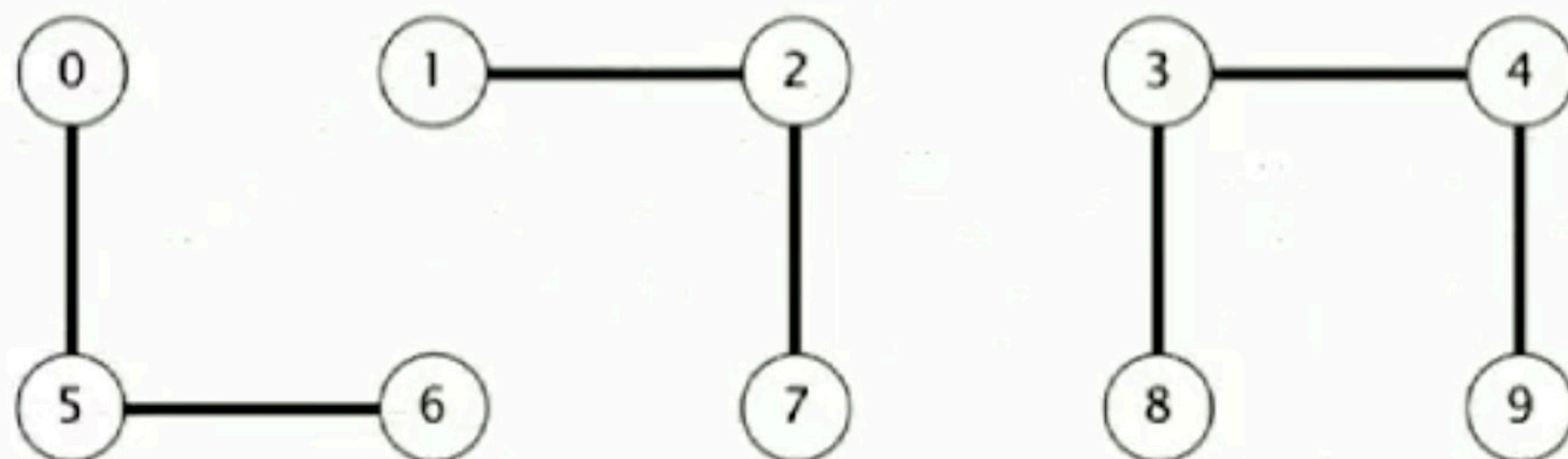
- Integer array `id[]` of size `N`.
- Interpretation: `p` and `q` are connected iff they have the same id.

if and only if



	0	1	2	3	4	5	6	7	8	9
id[]	0	1	1	8	8	0	0	1	8	8

0, 5 and 6 are connected
1, 2, and 7 are connected
3, 4, 8, and 9 are connected



Quick-union: Java implementation

```
public class QuickUnionUF  
{
```

```
    private int[] id;
```

```
    public QuickUnionUF(int N)
```

```
    {
```

```
        id = new int[N];
```

```
        for (int i = 0; i < N; i++) id[i] = i;
```

```
    }
```

```
    private int root(int i)
```

```
    {
```

```
        while (i != id[i]) i = id[i];
```

```
        return i;
```

```
    }
```

```
    public boolean connected(int p, int q)
```

```
    {
```

```
        return root(p) == root(q);
```

```
    }
```

```
    public void union(int p, int q)
```

```
    {
```

```
        int i = root(p)
```

```
        int j = root(q);
```

```
        id[i] = j;
```

```
    }
```

```
}
```

set id of each object to itself
(N array accesses)

chase parent pointers until reach root
(depth of i array accesses)

check if p and q have same root
(depth of p and q array accesses)

change root of p to point to root of q
(depth of p and q array accesses)

Quick-union is also too slow

Cost model. Number of array accesses (for read or write).

algorithm	initialize	union	find
quick-find	N	N	1
quick-union	N	$N \dagger$	N

← worst case

† includes cost of finding roots

Quick-find defect.

- Union too expensive (N array accesses).
- Trees are flat, but too expensive to keep them flat.

Quick-union defect.

- Trees can get tall.
- Find too expensive (could be N array accesses).

Common order-of-growth classifications

order of growth	name	typical code framework	description	example	$T(2N) / T(N)$
1	constant	<code>a = b + c;</code>	statement	add two numbers	1
$\log N$	logarithmic	<code>while (N > 1) { N = N / 2; ... }</code>	divide in half	binary search	~ 1
N	linear	<code>for (int i = 0; i < N; i++) { ... }</code>	loop	find the maximum	2
$N \log N$	linearithmic	[see mergesort lecture]	divide and conquer	mergesort	~ 2
N^2	quadratic	<code>for (int i = 0; i < N; i++) for (int j = 0; j < N; j++) { ... }</code>	double loop	check all pairs	4
N^3	cubic	<code>for (int i = 0; i < N; i++) for (int j = 0; j < N; j++) for (int k = 0; k < N; k++) { ... }</code>	triple loop	check all triples	8
2^N	exponential	[see combinatorial search lecture]	exhaustive search	check all subsets	$T(N)$

Practical implications of order-of-growth

growth rate	problem size solvable in minutes			
	1970s	1980s	1990s	2000s
1	any	any	any	any
log N	any	any	any	any
N	millions	tens of millions	hundreds of millions	billions
N log N	hundreds of thousands	millions	millions	hundreds of millions
N ²	hundreds	thousand	thousands	tens of thousands
N ³	hundred	hundreds	thousand	thousands
2 ^N	20	20s	20s	30

Bottom line. Need linear or linearithmic alg to keep pace with Moore's law.

Typical memory usage for objects in Java

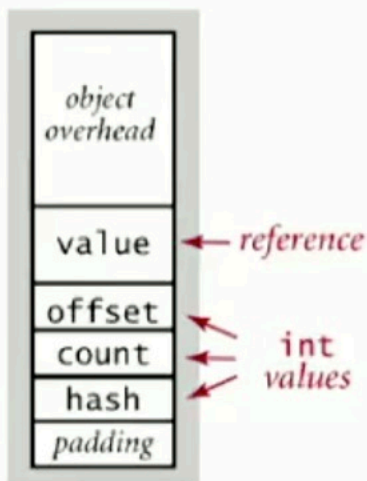
Object overhead. 16 bytes.

Reference. 8 bytes.

Padding. Each object uses a multiple of 8 bytes.

Ex 2. A virgin String of length N uses $\sim 2N$ bytes of memory.

```
public class String
{
    private char[] value;
    private int offset;
    private int count;
    private int hash;
    ...
}
```



16 bytes (object overhead)

8 bytes (reference to array)
2N + 24 bytes (char[] array)

4 bytes (int)

4 bytes (int)

4 bytes (int)

4 bytes (padding)

2N + 64 bytes

Typical memory usage for objects in Java

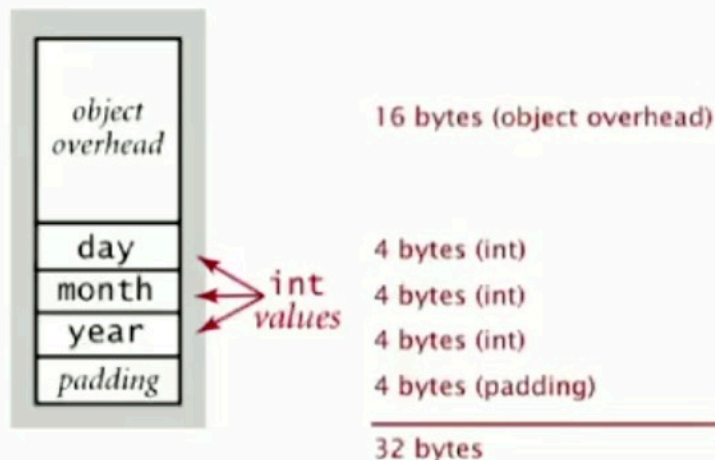
Object overhead. 16 bytes.

Reference. 8 bytes.

Padding. Each object uses a multiple of 8 bytes.

Ex 1. A Date object uses 32 bytes of memory.

```
public class Date
{
    private int day;
    private int month;
    private int year;
    ...
}
```



Typical memory usage summary

Total memory usage for a data type value:

- Primitive type: 4 bytes for int, 8 bytes for double, ...
- Object reference: 8 bytes.
- Array: 24 bytes + memory for each array entry.
- Object: 16 bytes + memory for each instance variable + 8 if inner class (for pointer to enclosing class).
- Padding: round up to multiple of 8 bytes.

Shallow memory usage: Don't count referenced objects.

Deep memory usage: If array entry or instance variable is a reference, add memory (recursively) for referenced object.

Cast of characters



Programmer needs to develop a working solution.



Student might play any or all of these roles someday.



Client wants to solve problem efficiently.



Theoretician wants to understand.



Basic **blocking and tackling** is sometimes necessary.
[this lecture]

Example: 3-SUM

3-SUM. Given N distinct integers, how many triples sum to exactly zero?

```
% more 8ints.txt
8
30 -40 -20 -10 40 0 10 5

% java ThreeSum 8ints.txt
4
```

	a[i]	a[j]	a[k]	sum
1	30	-40	10	0
2	30	-20	-10	0
3	-40	40	0	0
4	-10	0	10	0

Context. Deeply related to problems in computational geometry.

3-SUM: brute-force algorithm

```
public class ThreeSum
{
    public static int count(int[] a)
    {
        int N = a.length;
        int count = 0;
        for (int i = 0; i < N; i++)
            for (int j = i+1; j < N; j++)
                for (int k = j+1; k < N; k++)
                    if (a[i] + a[j] + a[k] == 0)
                        count++;
        return count;
    }

    public static void main(String[] args)
    {
        int[] a = In.readInts(args[0]);
        StdOut.println(count(a));
    }
}
```

← check each triple

← for simplicity, ignore
integer overflow

Dijkstra's two-stack algorithm

Value: push onto the value stack.

Operator: push onto the operator stack.

Left parenthesis: ignore.

Right parenthesis: pop operator and two values; push the result of applying that operator to those values onto the operand stack.



(1 + ((2 + 3) * (4 * 5)))

↑

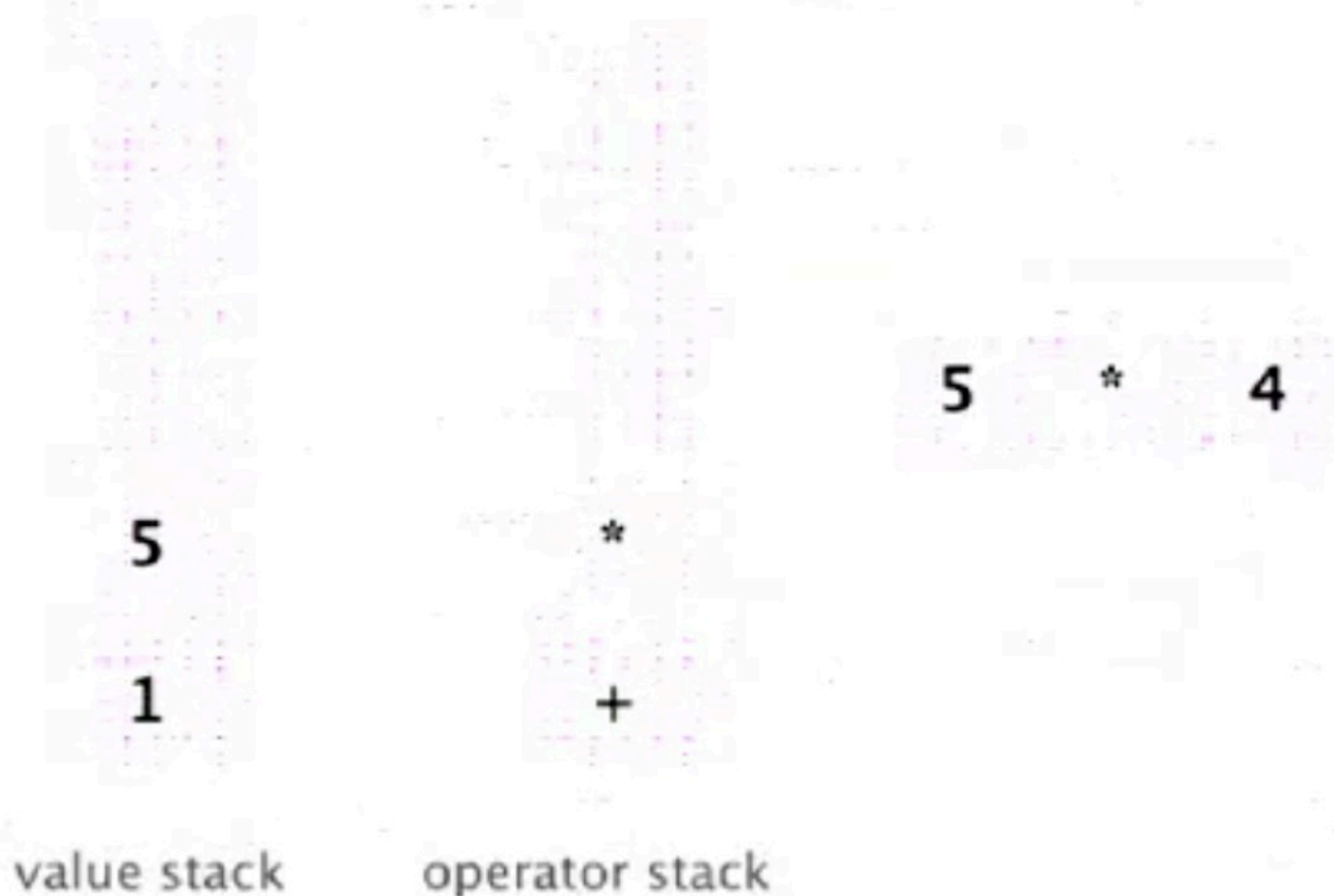
Dijkstra's two-stack algorithm

Value: push onto the value stack.

Operator: push onto the operator stack.

Left parenthesis: ignore.

Right parenthesis: pop operator and two values; push the result of applying that operator to those values onto the operand stack.



$((1 + ((2 + 3) * (4 * 5)))$

Implementing ccw

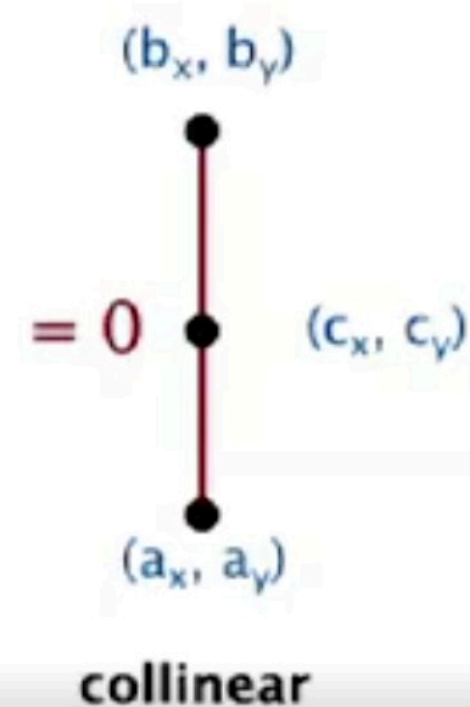
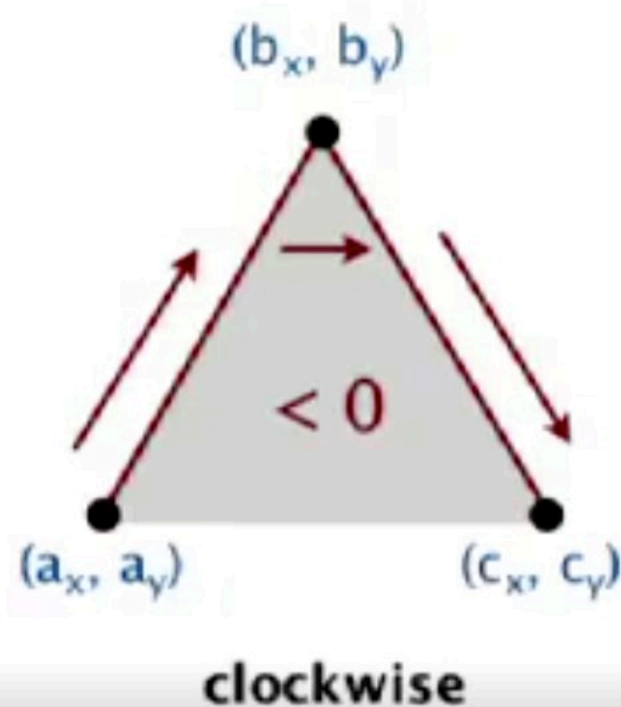
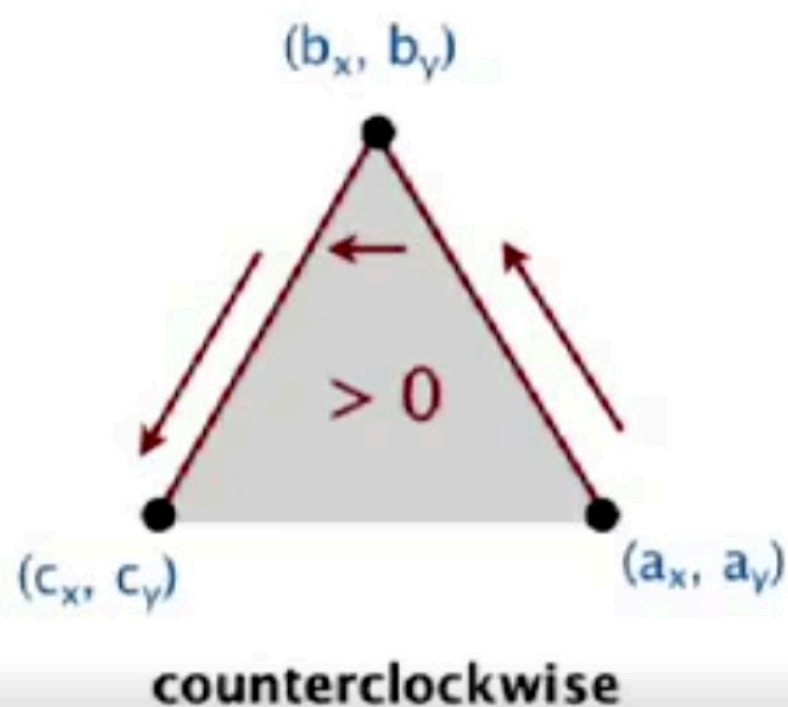
CCW. Given three points a , b , and c , is $a \rightarrow b \rightarrow c$ a counterclockwise turn?

- Determinant (or cross product) gives 2x signed area of planar triangle.

$$2 \times \text{Area}(a, b, c) = \begin{vmatrix} a_x & a_y & 1 \\ b_x & b_y & 1 \\ c_x & c_y & 1 \end{vmatrix} = (b_x - a_x)(c_y - a_y) - (b_y - a_y)(c_x - a_x)$$

$(b - a) \times (c - a)$

- If signed area > 0 , then $a \rightarrow b \rightarrow c$ is counterclockwise.
- If signed area < 0 , then $a \rightarrow b \rightarrow c$ is clockwise.
- If signed area $= 0$, then $a \rightarrow b \rightarrow c$ are collinear.



Immutable point data type

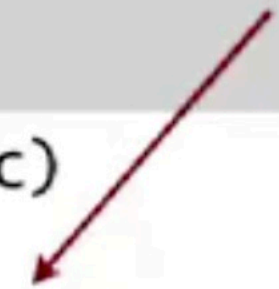
```
public class Point2D
{
    private final double x;
    private final double y;

    public Point(double x, double y)
    {
        this.x = x;
        this.y = y;
    }

    ...

    public static int ccw(Point a, Point b, Point c)
    {
        int area2 = (b.x-a.x)*(c.y-a.y) - (b.y-a.y)*(c.x-a.x);
        if (area2 < 0) return -1; // clockwise
        else if (area2 > 0) return +1; // counter-clockwise
        else return 0; // collinear
    }
}
```

danger of
floating-point
roundoff error



Graham scan: implementation

Simplifying assumptions. No three points on a line; at least 3 points.

```
Stack<Point2D> hull = new Stack<Point>();

Arrays.sort(p, Point2D.Y_ORDER);      ← p[0] is now point with lowest y-coordinate
Arrays.sort(p, p[0].BY_POLAR_ORDER); ← sort by polar angle with respect to p[0]

hull.push(p[0]); ← definitely on hull
hull.push(p[1]);

for (int i = 2; i < N; i++) {
    Point2D top = hull.pop();
    while (Point2D.ccw(hull.peek(), top, p[i]) <= 0)
        top = hull.pop();
    hull.push(top);
    hull.push(p[i]); ← add p[i] to putative hull
}
```

discard points that would
create clockwise turn
↓

Running time. $N \log N$ for sorting and linear for rest.

Pf. $N \log N$ for sorting; each point pushed and popped at most once.

Mergesort: practical improvements

Use insertion sort for small subarrays.

- Mergesort has too much overhead for tiny subarrays.
- Cutoff to insertion sort for ≈ 7 items.



```
private static void sort(Comparable[] a, Comparable[] aux, int lo, int hi)
{
    if (hi <= lo + CUTOFF - 1) Insertion.sort(a, lo, hi);
    int mid = lo + (hi - lo) / 2;
    sort (a, aux, lo, mid);
    sort (a, aux, mid+1, hi);
    merge(a, aux, lo, mid, hi);
}
```

Mergesort: practical improvements

Stop if already sorted.

- Is biggest item in first half $\leq$ smallest item in second half?
- Helps for partially-ordered arrays.

A B C D E F G H I J M N O P Q R S T U V

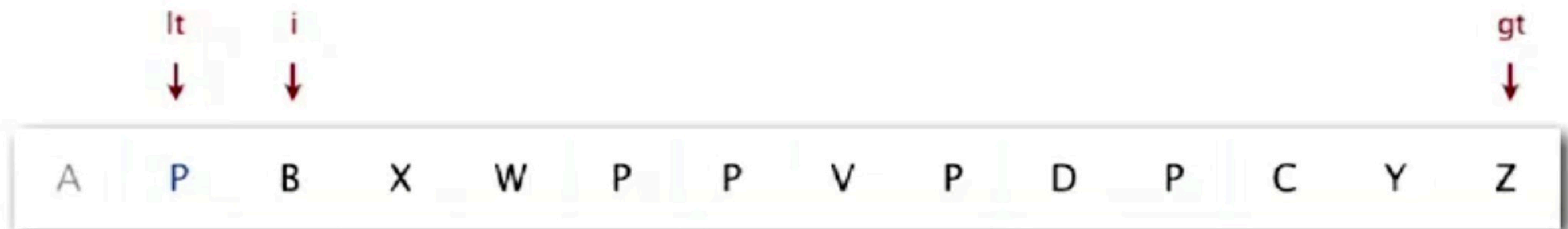
A B C D E F G H I J M N O P Q R S T U V



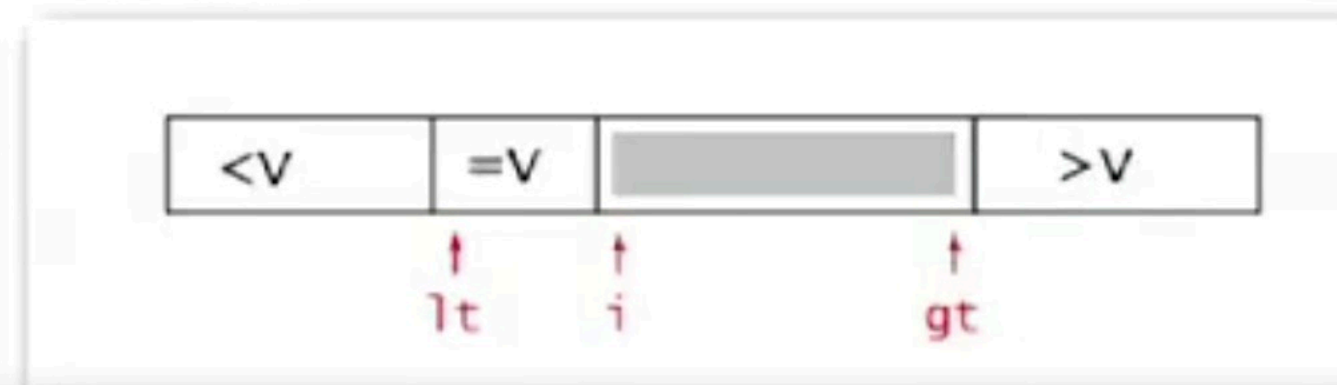
```
private static void sort(Comparable[] a, Comparable[] aux, int lo, int hi)
{
    if (hi <= lo) return;
    int mid = lo + (hi - lo) / 2;
    sort(a, aux, lo, mid);
    sort(a, aux, mid+1, hi);
    if (!less(a[mid+1], a[mid])) return;
    merge(a, aux, lo, mid, hi);
}
```


Dijkstra 3-way partitioning demo

- Let v be partitioning item $a[l_0]$.
- Scan i from left to right.
 - ($a[i] < v$): exchange $a[l_t]$ with $a[i]$; increment both l_t and i
 - ($a[i] > v$): exchange $a[gt]$ with $a[i]$; decrement gt
 - ($a[i] == v$): increment i

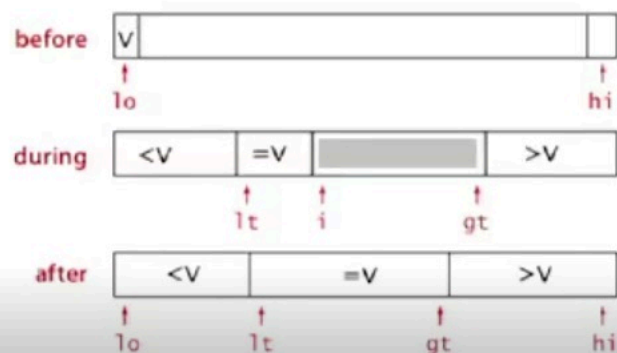


invariant



3-way quicksort: Java implementation

```
private static void sort(Comparable[] a, int lo, int hi)
{
    if (hi <= lo) return;
    int lt = lo, gt = hi;
    Comparable v = a[lo];
    int i = lo;
    while (i <= gt)
    {
        int cmp = a[i].compareTo(v);
        if      (cmp < 0) exch(a, lt++, i++);
        else if (cmp > 0) exch(a, i, gt--);
        else          i++;
    }
    sort(a, lo, lt - 1);
    sort(a, gt + 1, hi);
}
```



Sorting applications

Sorting algorithms are essential in a broad variety of applications:

- Sort a list of names.
- Organize an MP3 library.
- Display Google PageRank results.
- List RSS feed in reverse chronological order.

obvious applications

- Find the median.
- Binary search in a database.
- Identify statistical outliers.
- Find duplicates in a mailing list.

problems become easy once items
are in sorted order

- Data compression.
- Computer graphics.
- Computational biology.
- Load balancing on a parallel computer.

non-obvious applications

...

System sort: Which algorithm to use?

Many sorting algorithms to choose from:

Internal sorts.

- Insertion sort, selection sort, bubblesort, shaker sort.
- Quicksort, mergesort, heapsort, samplesort, shellsort.
- Solitaire sort, red-black sort, splay sort, Yaroslavskiy sort, psort, ...

External sorts. Poly-phase mergesort, cascade-merge, oscillating sort.

String/radix sorts. Distribution, MSD, LSD, 3-way string quicksort.

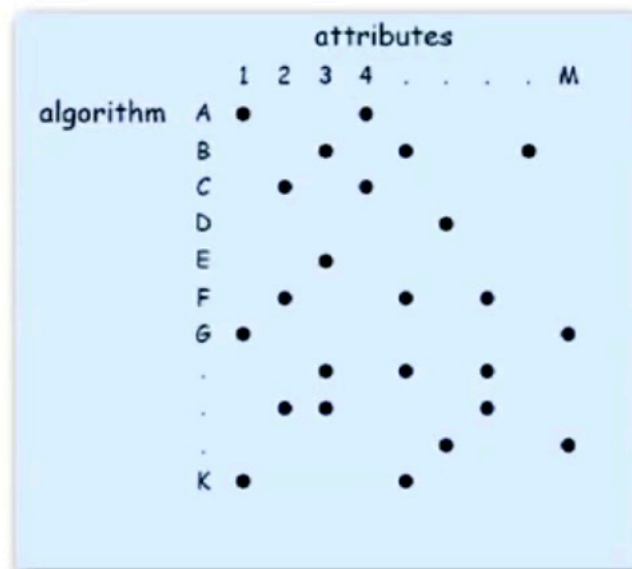
Parallel sorts.

- Bitonic sort, Batcher even-odd sort.
- Smooth sort, cube sort, column sort.
- GPU sort.

System sort: Which algorithm to use?

Applications have diverse attributes.

- Stable?
- Parallel?
- Deterministic?
- Keys all distinct?
- Multiple key types?
- Linked list or arrays?
- Large or small items?
- Is your array randomly ordered?
- Need guaranteed performance?



many more combinations of attributes than algorithms

Elementary sort may be method of choice for some combination.

Cannot cover **all** combinations of attributes.

Q. Is the system sort good enough?

A. Usually.

Sorting summary

	inplace?	stable?	worst	average	best	remarks
selection	✓		$N^2 / 2$	$N^2 / 2$	$N^2 / 2$	N exchanges
insertion	✓	✓	$N^2 / 2$	$N^2 / 4$	N	use for small N or partially ordered
shell	✓		?	?	N	tight code, subquadratic
merge		✓	$N \lg N$	$N \lg N$	$N \lg N$	$N \log N$ guarantee, stable
quick	✓		$N^2 / 2$	$2 N \ln N$	$N \lg N$	$N \log N$ probabilistic guarantee fastest in practice
3-way quick	✓		$N^2 / 2$	$2 N \ln N$	N	improves quicksort in presence of duplicate keys
???	✓	✓	$N \lg N$	$N \lg N$	$N \lg N$	holy sorting grail

Sorting algorithms: summary

	inplace?	stable?	worst	average	best	remarks
selection	x		$N^2 / 2$	$N^2 / 2$	$N^2 / 2$	N exchanges
insertion	x	x	$N^2 / 2$	$N^2 / 4$	N	use for small N or partially ordered
shell	x		?	?	N	tight code, subquadratic
quick	x		$N^2 / 2$	$2 N \ln N$	$N \lg N$	$N \log N$ probabilistic guarantee fastest in practice
3-way quick	x		$N^2 / 2$	$2 N \ln N$	N	improves quicksort in presence of duplicate keys
merge		x	$N \lg N$	$N \lg N$	$N \lg N$	$N \log N$ guarantee, stable
heap	x		$2 N \lg N$	$2 N \lg N$	$N \lg N$	$N \log N$ guarantee, in-place
???	x	x	$N \lg N$	$N \lg N$	$N \lg N$	holy sorting grail

Priority queue

Collections. Insert and delete items. Which item to delete?

Stack. Remove the item most recently added.

Queue. Remove the item least recently added.

Randomized queue. Remove a random item.

Priority queue. Remove the **largest** (or **smallest**) item.

<i>operation</i>	<i>argument</i>	<i>return value</i>
<i>insert</i>	P	
<i>insert</i>	Q	
<i>insert</i>	E	
<i>remove max</i>		Q
<i>insert</i>	X	
<i>insert</i>	A	
<i>insert</i>	M	
<i>remove max</i>		X
<i>insert</i>	P	
<i>insert</i>	L	
<i>insert</i>	E	
<i>remove max</i>		P

Priority queue applications

- Event-driven simulation. [customers in a line, colliding particles]
- Numerical computation. [reducing roundoff error]
- Data compression. [Huffman codes]
- Graph searching. [Dijkstra's algorithm, Prim's algorithm]
- Number theory. [sum of powers]
- Artificial intelligence. [A* search]
- Statistics. [maintain largest M values in a sequence]
- Operating systems. [load balancing, interrupt handling]
- Discrete optimization. [bin packing, scheduling]
- Spam filtering. [Bayesian spam filter]

Generalizes: stack, queue, randomized queue.

Priority queue client example

Challenge. Find the largest M items in a stream of N items.

- Fraud detection: isolate \$\$ transactions.
- File maintenance: find biggest files or directories.

N huge, M large

Constraint. Not enough memory to store N items.

```
% more tinyBatch.txt
Turing      6/17/1990    644.08
vonNeumann  3/26/2002    4121.85
Dijkstra    8/22/2007    2678.40
vonNeumann  1/11/1999    4409.74
Dijkstra    11/18/1995    837.42
Hoare       5/10/1993    3229.27
vonNeumann  2/12/1994    4732.35
Hoare       8/18/1992    4381.21
Turing      1/11/2002     66.10
Thompson    2/27/2000    4747.08
Turing      2/11/1991    2156.86
Hoare       8/12/2003    1025.70
vonNeumann  10/13/1993    2520.97
Dijkstra    9/10/2000     708.95
Turing      10/12/1993    3532.36
Hoare       2/10/2005    4050.20
```

```
% java TopM 5 < tinyBatch.txt
Thompson    2/27/2000    4747.08
vonNeumann  2/12/1994    4732.35
vonNeumann  1/11/1999    4409.74
Hoare       8/18/1992    4381.21
vonNeumann  3/26/2002    4121.85
```

sort key

Binary heap representations

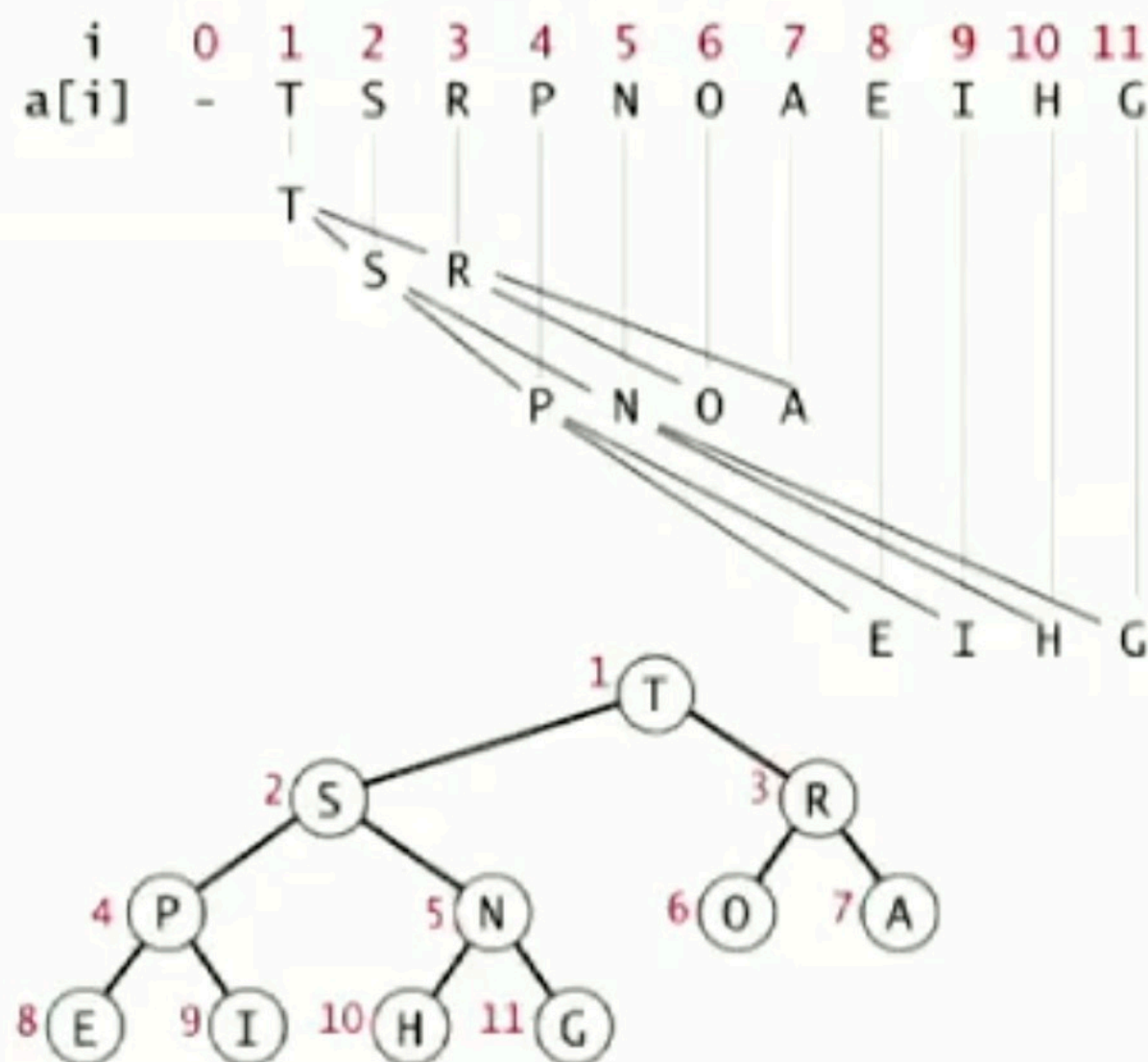
Binary heap. Array representation of a heap-ordered complete binary tree.

Heap-ordered binary tree.

- Keys in nodes.
- Parent's key no smaller than children's keys.

Array representation.

- Indices start at 1.
- Take nodes in **level** order.
- No explicit links needed!



Heap representations

Equality test

All Java classes inherit a method `equals()`.

Java requirements. For any references `x`, `y` and `z`:

- Reflexive: `x.equals(x)` is true.
- Symmetric: `x.equals(y)` iff `y.equals(x)`.
- Transitive: if `x.equals(y)` and `y.equals(z)`, then `x.equals(z)`.
- Non-null: `x.equals(null)` is false.

} equivalence
relation

do `x` and `y` refer to
the same object?

Default implementation. `(x == y)`

Customized implementations. Integer, Double, String, File, URL, ...

User-defined implementations. Some care needed.

Implementing equals for user-defined types

Seems easy, but requires some care.

typically unsafe to use `equals()` with inheritance
(would violate symmetry)

```
public final class Date implements Comparable<Date>
{
    private final int month;
    private final int day;
    private final int year;
    ...

    public boolean equals(Object y)
    {
        if (y == this) return true;

        if (y == null) return false;

        if (y.getClass() != this.getClass())
            return false;

        Date that = (Date) y;
        if (this.day != that.day ) return false;
        if (this.month != that.month) return false;
        if (this.year != that.year ) return false;
        return true;
    }
}
```

must be `Object`.
Why? Experts still debate.

optimize for true object equality

check for `null`

objects must be in the same class
(religion: `getClass()` vs. `instanceof`)

cast is guaranteed to succeed

check that all significant
fields are the same

Equals design

"Standard" recipe for user-defined types.

- Optimization for reference equality.
- Check against `null`.
- Check that two objects are of the same type and cast.
- Compare each significant field:
 - if field is a primitive type, use `==`
 - if field is an object, use `equals()`  apply rule recursively
 - if field is an array, apply to each entry  alternatively, use `Arrays.equals(a, b)` or `Arrays.deepEquals(a, b)`, but not `a.equals(b)`

Best practices.

- No need to use calculated fields that depend on other fields.
- Compare fields mostly likely to differ first.
- Make `compareTo()` consistent with `equals()`.

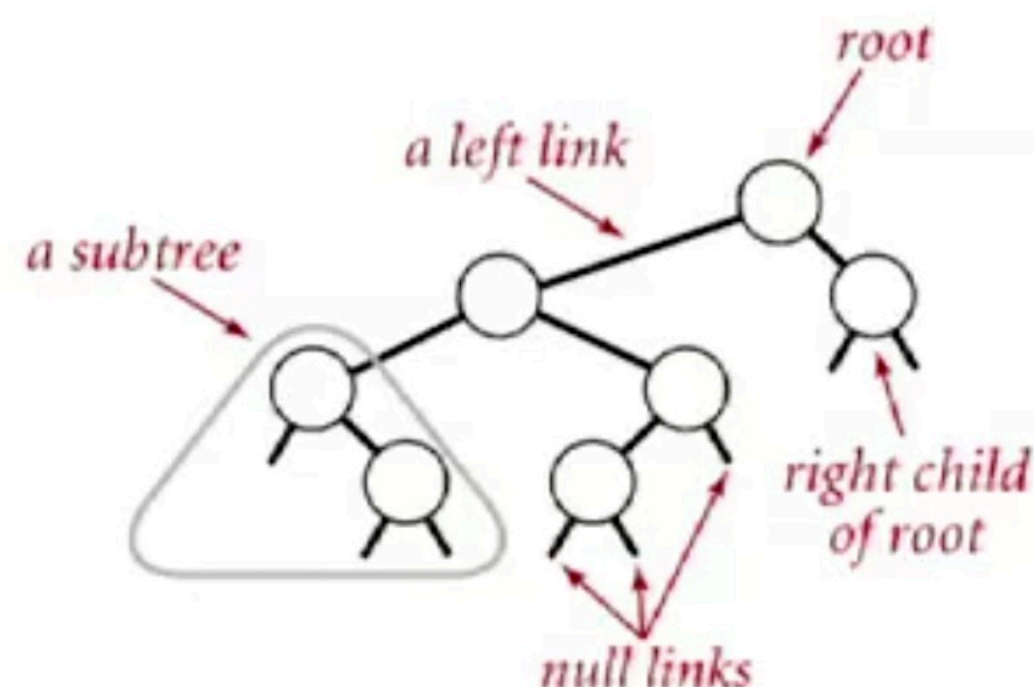
 `x.equals(y)` if and only if `(x.compareTo(y) == 0)`

Binary search trees

Definition. A BST is a **binary tree** in **symmetric order**.

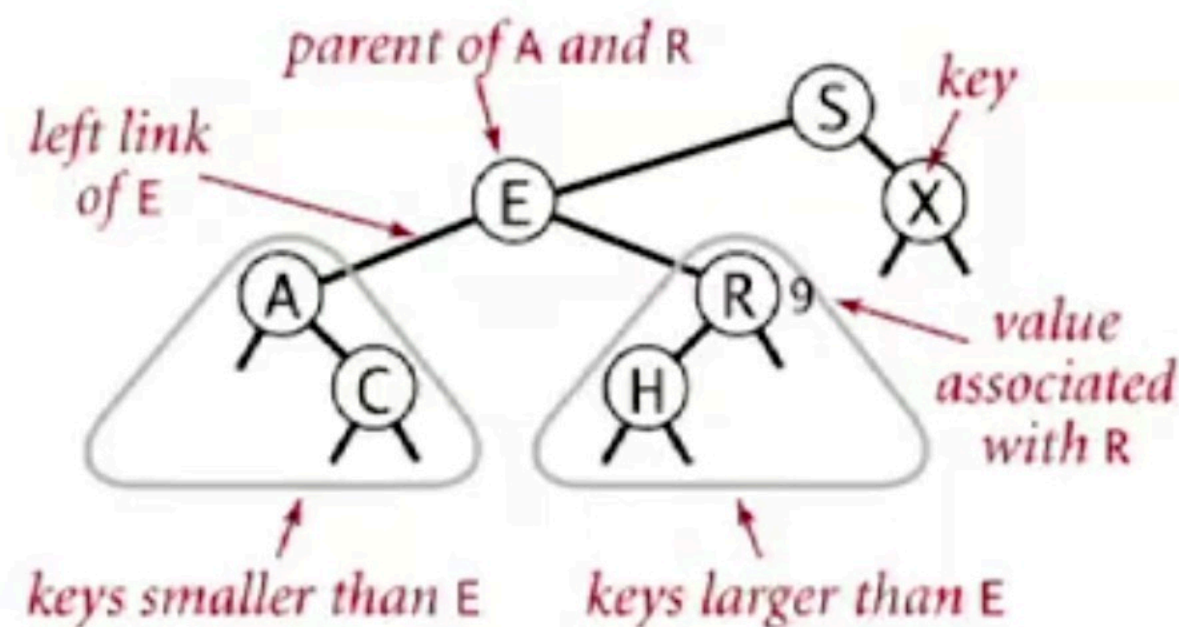
A binary tree is either:

- Empty.
- Two disjoint binary trees (left and right).



Symmetric order. Each node has a key, and every node's key is:

- Larger than all keys in its left subtree.
- Smaller than all keys in its right subtree.



BST representation in Java

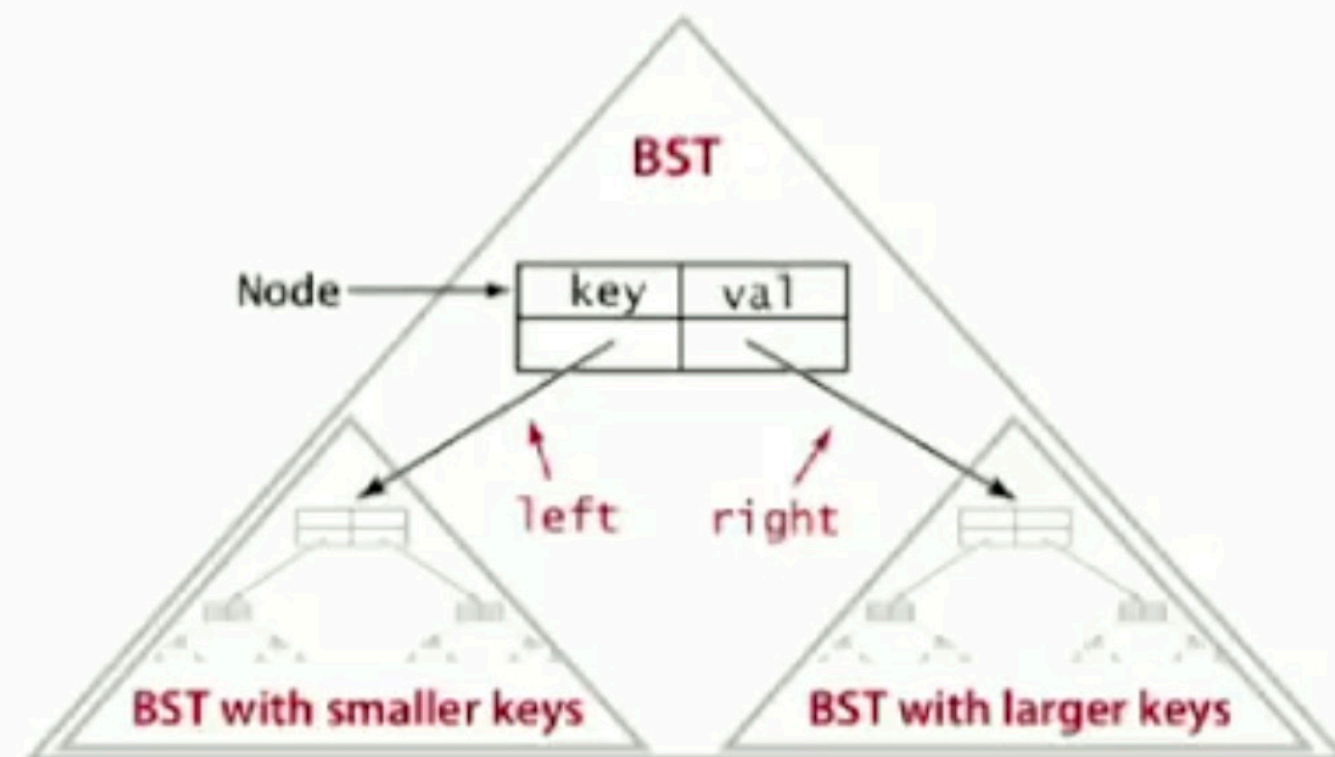
Java definition. A BST is a reference to a root Node.

A Node is comprised of four fields:

- A Key and a Value.
- A reference to the left and right subtree.


smaller keys larger keys

```
private class Node
{
    private Key key;
    private Value val;
    private Node left, right;
    public Node(Key key, Value val)
    {
        this.key = key;
        this.val = val;
    }
}
```

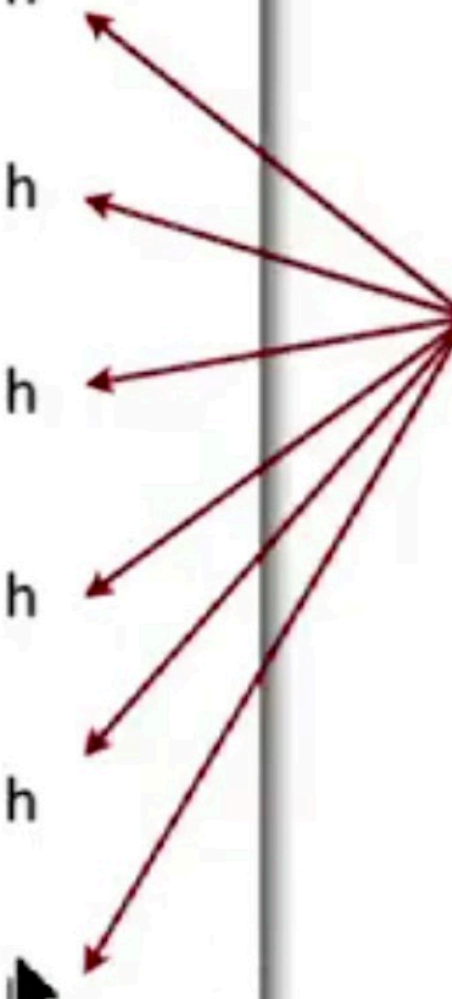


Binary search tree

Key and Value are generic types; Key is Comparable

BST: ordered symbol table operations summary

	sequential search	binary search	BST
search	N	$\lg N$	h
insert	1	N	h
min / max	N	1	h
floor / ceiling	N	$\lg N$	h
rank	N	$\lg N$	h
select	N	1	1
ordered iteration	$N \lg N$	N	N



h = height of BST
(proportional to $\lg N$
if keys inserted in random order)

order of growth of running time of ordered symbol table operations

ST implementations: summary

implementation	guarantee			average case			ordered iteration?	operations on keys
	search	insert	delete	search hit	insert	delete		
sequential search (linked list)	N	N	N	N/2	N	N/2	no	equals()
binary search (ordered array)	$\lg N$	N	N	$\lg N$	N/2	N/2	yes	compareTo()
BST	N	N	N	$1.39 \lg N$	$1.39 \lg N$	???	yes	compareTo()

File Edit View Search Terminal Help

```
#' <RED_HAT_PASSWORD>' ps -e | grep kube
# ps -e | grep etcd
# kubectl get service (Not specific to one container)
# curl http://10.254.73.89/
# curl http://10.254.73.89/cgi-bin/action
# kubectl get endpoints (shows IP/port for each container)
# kubectl get replicationcontroller (shows two desired, two running each)
# docker ps | grep dbforweb
# docker stop <ID>
# docker rm <ID>
# docker ps | grep dbforweb
# kubectl scale replicationcontroller --replicas=4 webserver-controller
# kubectl get replicationcontroller
# etcdctl ls /registry
# etcdctl ls /registry/ranges
# etcdctl get /registry/ranges/servicenodeports
```

File Edit View Search Terminal Help

```
$ sudo yum install git
$ oc login -u developer
$ oc new-app https://github.com/openshift/nodejs-ex -l name=myapp
    Get content from github. Label all resources as myapp.
$ oc logs -f bc/nodejs-ex
    Follow along with logs.
$ oc expose svc/nodejs-ex
    Expose route externally
$ minishift openshift service nodejs-ex --in-browser
    Open the URL to the service in a Web browser

Select Builds -> See the container image created
Applications -> See the active image
$ minishift ssh
$ sudo docker ps | grep nodejs
    See the nodejs image running inside the minishift virtual machine
```

File Edit View Search Terminal Help

```
$ sudo yum install cockpit cockpit-dashboard cockpit-docker
```

If firewalld is running

```
$ firewall-cmd --add-port=9090/tcp
```

```
$ firewall-cmd --permanent --add-port=9090/tcp
```

```
$ sudo systemctl enable cockpit.socket
```

```
$ sudo systemctl start cockpit.socket
```

In Web browser, open localhost:9090

Log into Cockpit console

Select -> Containers

See running containers and images

Select -> Get new image -> Search for rhel7-init -> Download

Select Images -> docker.io/cricket/hangman:latest -> run hangman in terminal

Graph applications

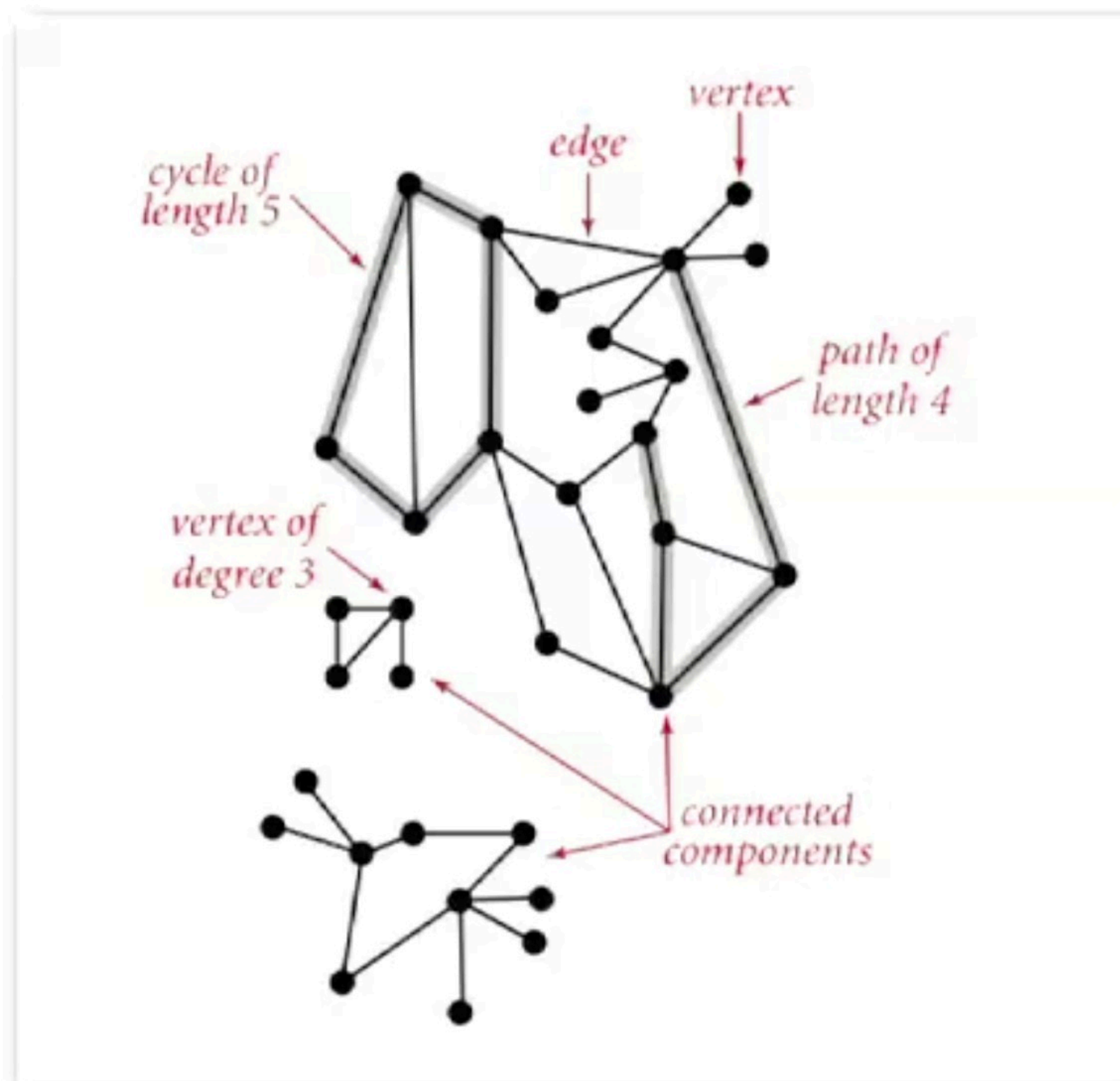
graph	vertex	edge
communication	telephone, computer	fiber optic cable
circuit	gate, register, processor	wire
mechanical	joint	rod, beam, spring
financial	stock, currency	transactions
transportation	street intersection, airport	highway, airway route
internet	class C network	connection
game	board position	legal move
social relationship	person, actor	friendship, movie cast
neural network	neuron	synapse
protein network	protein	protein-protein interaction
molecule	atom	bond

Graph terminology

Path. Sequence of vertices connected by edges.

Cycle. Path whose first and last vertices are the same.

Two vertices are **connected** if there is a path between them.



Some graph-processing problems

Path. Is there a path between s and t ?

Shortest path. What is the shortest path between s and t ?

Cycle. Is there a cycle in the graph?

Euler tour. Is there a cycle that uses each edge exactly once?

Hamilton tour. Is there a cycle that uses each vertex exactly once.

Connectivity. Is there a way to connect all of the vertices?

MST. What is the best way to connect all of the vertices?

Biconnectivity. Is there a vertex whose removal disconnects the graph?

Planarity. Can you draw the graph in the plane with no crossing edges

Graph isomorphism. Do two adjacency lists represent the same graph?

Challenge. Which of these problems are easy? difficult? intractable?

Graph API

```
public class Graph
```

```
    Graph(int V)
```

create an empty graph with V vertices

```
    Graph(In in)
```

create a graph from input stream

```
    void addEdge(int v, int w)
```

add an edge v-w

```
    Iterable<Integer> adj(int v)
```

vertices adjacent to v

```
    int V()
```

number of vertices

```
    int E()
```

number of edges

```
    String toString()
```

string representation

```
In in = new In(args[0]);  
Graph G = new Graph(in);
```

← *read graph from
input stream*

```
for (int v = 0; v < G.V(); v++)  
    for (int w : G.adj(v))  
        StdOut.println(v + "-" + w);
```

← *print out each
edge (twice)*

Typical graph-processing code

compute the degree of v

```
public static int degree(Graph G, int v)
{
    int degree = 0;
    for (int w : G.adj(v)) degree++;
    return degree;
}
```

compute maximum degree

```
public static int maxDegree(Graph G)
{
    int max = 0;
    for (int v = 0; v < G.V(); v++)
        if (degree(G, v) > max)
            max = degree(G, v);
    return max;
}
```

compute average degree

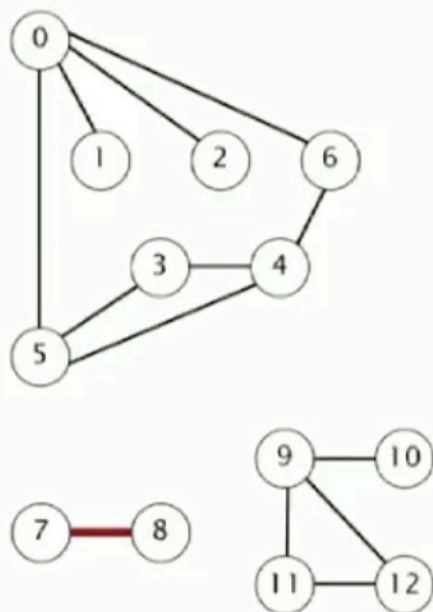
```
public static double averageDegree(Graph G)
{ return 2.0 * G.E() / G.V(); }
```

count self-loops

```
public static int numberOfSelfLoops(Graph G)
{
    int count = 0;
    for (int v = 0; v < G.V(); v++)
        for (int w : G.adj(v))
            if (v == w) count++;
    return count/2; // each edge counted twice
}
```

Set-of-edges graph representation

Maintain a list of the edges (linked list or array).

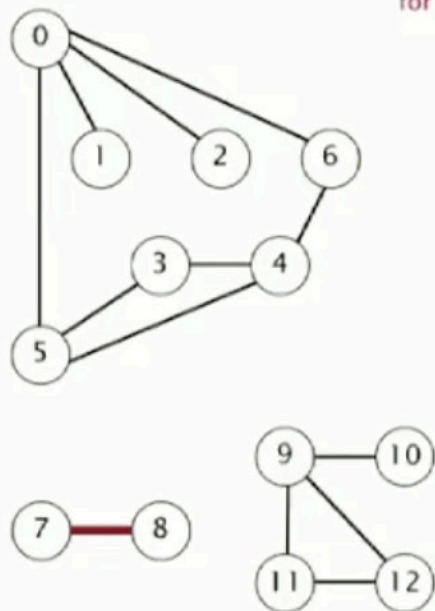


0	1
0	2
0	5
0	6
3	4
3	5
4	5
4	6
7	8
9	10
9	11
9	12
11	12

Adjacency-matrix graph representation

Maintain a two-dimensional V -by- V boolean array;

for each edge $v-w$ in graph: $\text{adj}[v][w] = \text{adj}[w][v] = \text{true}$.

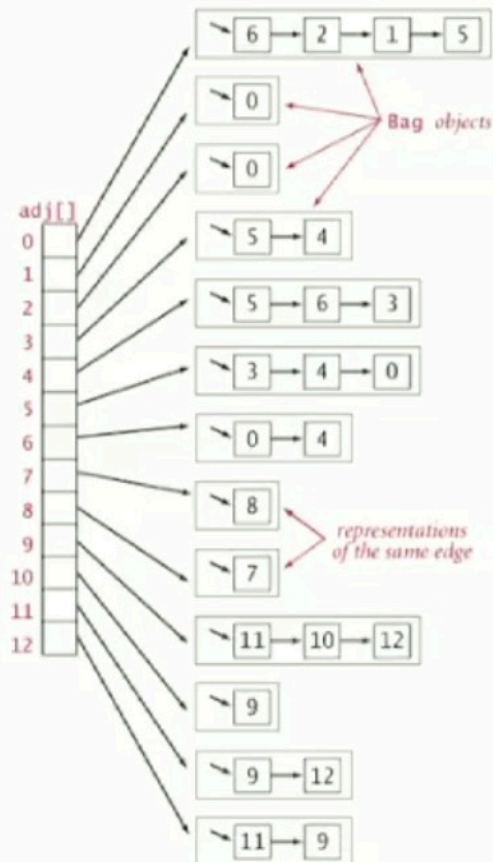
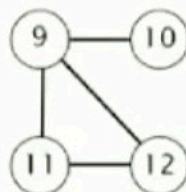
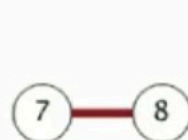
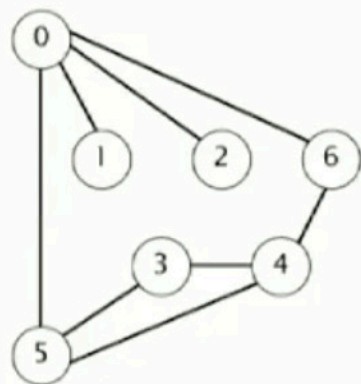


two entries
for each edge

	0	1	2	3	4	5	6	7	8	9	10	11	12
0	0	1	1	0	0	1	1	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0	0	0	0
2	1	0	0	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	1	1	0	0	0	0	0	0	0
4	0	0	0	1	0	1	1	0	0	0	0	0	0
5	1	0	0	1	1	0	0	0	0	0	0	0	0
6	1	0	0	0	1	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0	1	0	0	0	0
8	0	0	0	0	0	0	0	1	0	0	0	0	0
9	0	0	0	0	0	0	0	0	0	0	1	1	1
10	0	0	0	0	0	0	0	0	0	1	0	0	0
11	0	0	0	0	0	0	0	0	0	1	0	0	1
12	0	0	0	0	0	0	0	0	0	1	0	1	0

Adjacency-list graph representation

Maintain vertex-indexed array of lists.



Adjacency-list graph representation: Java implementation

```
public class Graph
```

```
{
```

```
    private final int V;
```

```
    private Bag<Integer>[] adj;
```

← adjacency lists
(using Bag data type)

```
    public Graph(int V)
```

```
    {
```

```
        this.V = V;
```

```
        adj = (Bag<Integer>[]) new Bag[V];
```

```
        for (int v = 0; v < V; v++)
```

```
            adj[v] = new Bag<Integer>();
```

```
    }
```

← create empty graph
with V vertices

```
    public void addEdge(int v, int w)
```

```
    {
```

```
        adj[v].add(w);
```

```
        adj[w].add(v);
```

```
    }
```

← add edge v-w
(parallel edges allowed)

```
    public Iterable<Integer> adj(int v)
```

```
    { return adj[v]; }
```

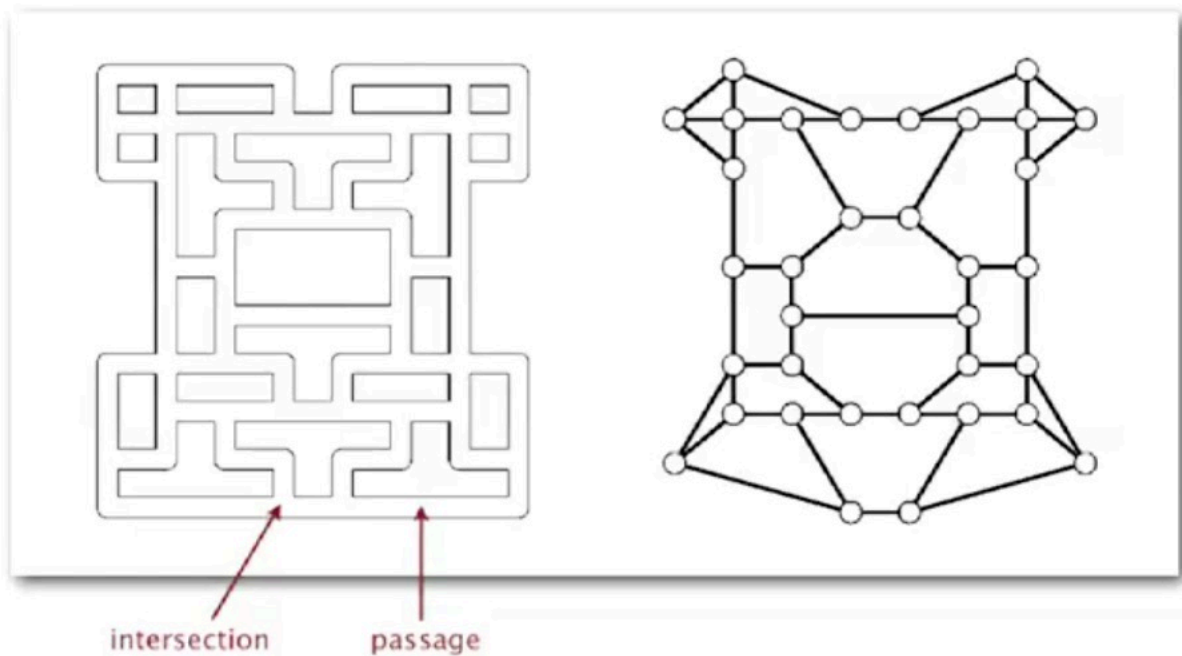
```
}
```

← iterator for vertices adjacent to v

Maze exploration

Maze graph.

- Vertex = intersection.
- Edge = passage.



Goal. Explore every intersection in the maze.

Depth-first search

Goal. Systematically search through a graph.

Idea. Mimic maze exploration.

DFS (to visit a vertex v)

Mark v as visited.

Recursively visit all unmarked
vertices w adjacent to v .

Typical applications.

- Find all vertices connected to a given source vertex.
- Find a path between two vertices.

Design pattern for graph processing

Design pattern. Decouple graph data type from graph processing.

- Create a Graph object.
- Pass the Graph to a graph-processing routine.
- Query the graph-processing routine for information.

```
public class Paths
```

```
    Paths(Graph G, int s)
```

find paths in G from source s

```
    boolean hasPathTo(int v)
```

is there a path from s to v?

```
    Iterable<Integer> pathTo(int v)
```

path from s to v; null if no such path

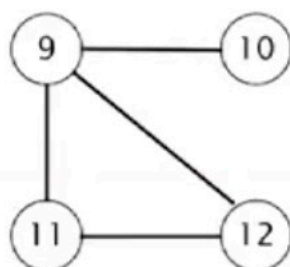
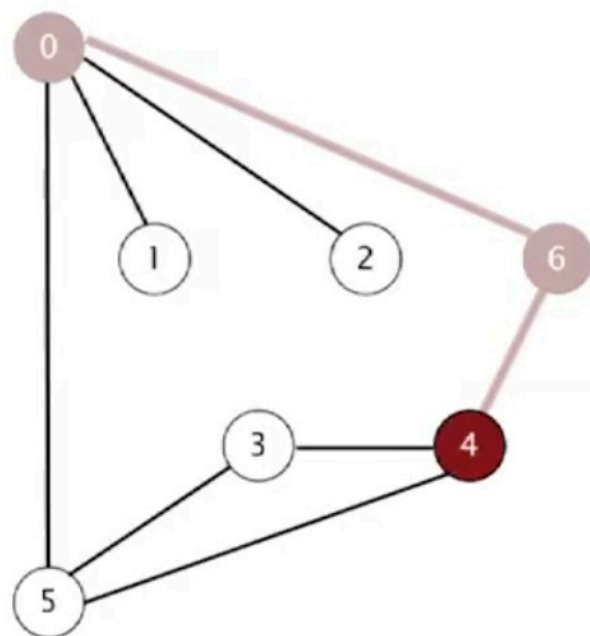
```
Paths paths = new Paths(G, s);  
for (int v = 0; v < G.V(); v++)  
    if (paths.hasPathTo(v))  
        StdOut.println(v);
```

← print all vertices
connected to s

Depth-first search demo

To visit a vertex v :

- Mark vertex v as visited.
- Recursively visit all unmarked vertices adjacent to v .



v	marked[]	edgeTo[v]
0	T	-
1	F	-
2	F	-
3	F	-
4	T	6
5	F	-
6	T	0
7	F	-
8	F	-
9	F	-
10	F	-
11	F	-
12	F	-

visit 4: check 5, check 6 and check 3

Depth-first search

Goal. Find all vertices connected to s (and a corresponding path).

Idea. Mimic maze exploration.

Algorithm.

- Use recursion (ball of string).
- Mark each visited vertex (and keep track of edge taken to visit it).
- Return (retrace steps) when no unvisited options.

Data structures.

- `boolean[] marked` to mark visited vertices.
- `int[] edgeTo` to keep tree of paths.
(`edgeTo[w] == v`) means that edge $v-w$ taken to visit w for first time